RUN GUIDE / CUDA TERM PROJECT - V2

Exact 1-Nearest-Neighbor Classification on UCI Forest Covertype

Includes two CPU serial baselines (naive and early-abandonment), two CUDA GPU kernels (baseline and SoA-optimized), public-dataset preparation, benchmark scripts, PowerPoint presentation, and technical report.

1. Requirements

Python 3.9+; NumPy and Matplotlib; a C++17 compiler. For the CUDA version, a compatible NVIDIA GPU, driver, and CUDA Toolkit with nvcc are required. Intel-only integrated graphics do not run CUDA.

2. Download and prepare the real dataset

```
python -m pip install -r requirements.txt
python scripts/prepare_data.py
```

Creates data/covertype.bin from the public UCI Covertype dataset (581,012 rows, 54 features). Default split: 572,820 references and 8,192 held-out queries. Features are standardized using reference-only statistics. If the automatic download fails, place covtype.data.gz in data/ and rerun.

Public source: https://archive.ics.uci.edu/dataset/31/covertype (Blackard, 1998; CC BY 4.0)

3. Compile on Linux / WSL

```
make cpu
make gpu
```

Or compile manually:

```
mkdir -p build
g++ -O3 -std=c++17 -ffp-contract=off src/cpu_only.cpp -o build/knn_cpu
nvcc -O3 -std=c++17 --fmad=false -Xcompiler -ffp-contract=off \
  src/main.cu -o build/knn_cuda
```

4. Compile on Windows Developer PowerShell

```
New-Item -ItemType Directory -Force build
nvcc -O3 -std=c++17 --fmad=false src/main.cu -o build/knn_cuda.exe
```

CUDA Toolkit and a compatible MSVC C++ toolchain must be configured. If nvcc is unavailable locally, follow docs/COLAB_GPU_TH.md for Google Colab GPU instructions (availability varies).

TESTS, BENCHMARKS & SUBMISSION

Run correctness checks before attempting GPU performance measurements.

5. Correctness testing

```
python scripts/make_tiny_fixture.py
./build/knn_cpu --data data/tiny.bin --queries 3 --compare-naive
python scripts/test_correctness.py ./build/knn_cpu
./build/knn_cuda --data data/tiny.bin --queries 3 --gpu-baseline
```

The tiny fixture has expected nearest-reference IDs [0, 3, 4]. A separate NumPy implementation checks the CPU algorithm. CUDA execution checks GPU predictions against the serial CPU outputs; any index mismatches must be investigated.

6. Run the actual large-data benchmark

```
python scripts/benchmark.py --queries 32 128 512 2048 --gpu-baseline
```

The script produces results/benchmark.csv, results/runtime.png, and results/speedup.png. The CPU naive reference is additionally measured for the smallest query count. You can run individual CUDA experiments using:

```
./build/knn_cuda --data data/covertype.bin --queries 32 \
  --gpu-baseline --cpu-naive --repeats 3
```

7. Understand measured time and speedup

Metric	Meaning
CPU ms	One sequential early-abandonment exact 1-NN pass
GPU kernel ms	Median of repeated warmed-up CUDA event timings; two kernels
GPU end-to-end ms	One GPU pass incl. SoA conversion, H2D, kernels and D2H
Speedup	CPU milliseconds / corresponding GPU milliseconds
Index mismatches	Different nearest-reference row ID vs CPU; target = 0

8. Fill the presentation using authentic measurements

```
python scripts/fill_results_slide.py --csv results/benchmark.csv \
  --pptx docs/CUDA_Covertype_Presentation.pptx
```

Creates docs/CUDA_Covertype_Presentation_MEASURED.pptx. The measured-results placeholder is on slide 12 of the 15-slide deck. Do not submit placeholder dashes as measured speedup. Save GPU model, CPU model, CUDA version and benchmark flags in results/hardware.txt.

Requirement: show CUDA beats the sequential version on large input. Submit GitHub link, source code, run guide, and presentation. Due Nov 4, 2026 at 23:59; presentation Nov 5, 2026.